VYORA

Small-Business Billing & Inventory Platform

V1 Release Gate & Roadmap Brief
Document Version: 2.2 (October 2026)
Target Audience: Senior Developer & QA Agents

Project Manager Mission Statement: Vyora is designed to provide Indian MSMEs with a rock-solid, production-grade business billing, inventory, and ledger application inspired by market leaders such as Vyapar, but built with clean web architecture, multi-tenant cloud persistence, and strict accounting invariants. This document serves as the authoritative release plan, feature matrix, and handoff protocol for Senior Developer and QA engineering agents.

1. Version 1 Release Goals: Planned vs. Achieved vs. Extra Scope

The core objective of Version 1 is delivering a dependable, connected daily-use commercial workflow rather than an isolated UI prototype. All calculations, stock adjustments, and ledger postings are authoritative on the backend.

Module / Req	Original V1 Specification	Implementation Status & Invariants	Extra Capabilities Accomplished
Auth & Tenant (V1-01, V1-02)	JWT login/register, business onboarding, GST profile, secure token expiration.	**DELIVERED & VERIFIED** Auto-tenant discovery, multi-tenant isolation via `x-business-id`, bcrypt hashing.	Session persistence recovery, sanitization against `undefined` header injection.
Access Control (V1-03, V1-18, V1-19)	Owner vs Staff role enforcement on backend API and UI menus.	**DELIVERED & VERIFIED** `RolesGuard` blocks staff mutations with 403; primary owner is immutable.	**Live Roles Switchboard:** Dedicated `/account?tab=ROLES` with granular module permission matrix.
Catalog Master (V1-04)	Products & services, SKU, unit of measure, GST tax categories, opening stock.	**DELIVERED & VERIFIED** Calculations in integer minor units (paisa); duplicate SKU prevention.	Unit filter, tax slab filter, low-stock threshold badges, search debounce (300ms).
Parties Master (V1-05)	Customers and suppliers, contact info, state code, GSTIN, opening balance ledger.	**DELIVERED & VERIFIED** Strict tenant scoping; balance tracking with real-time receivables/payables.	State filter, balance due filter, integrated party ledger modal with direct payment links.
Sales Invoicing (V1-06, V1-07, V1-14)	Draft/finalize invoice lifecycle, server-side tax math, unique number, A4 PDF/print.	**DELIVERED & VERIFIED** Draft to finalized state engine; stock deducted upon finalization; immutable documents.	Authoritative live calculation preview, print-optimized stylesheet, cancellation reason audit.
Inventory Ledger (V1-08)	Traceable movements log, sales reduce stock, purchases increase stock, audit logs.	**DELIVERED & VERIFIED** Idempotent stock mutations; adjustment reasons (Damaged, Recount, Return, Expiry).	Negative stock validation guard, inventory movements filter by date and movement type.
Payments & Vouchers (V1-09)	Full and partial payments, method (Cash, Bank, UPI, Cheque), reference/UTR.	**DELIVERED & VERIFIED** Invoice balance reconciliation, unique idempotency keys to prevent double-recording.	Payment-in (Receipt) & Payment-out (Payment) voucher types, reference number validation.
Purchases (V1-10)	Supplier purchase bill entry, line items, taxes, stock increment, accounts payable.	**DELIVERED & VERIFIED** Purchases immediately increment physical stock and record supplier liability.	Custom `DatePicker` calendar, dynamic purchase bill tax calculation, zero-product catalog card.
Expenses (V1-11)	Direct & indirect expense tracking, categories, payment methods, notes.	**DELIVERED & VERIFIED** Tenant-scoped expense entries; date range filtering and category breakdowns.	Instant category summary aggregation, method breakdown (UPI, Cash, Bank).
Reports & Dashboard (V1-12, V1-13)	Sales/Purchase registers, stock valuation summary, customer dues, sales charts.	**DELIVERED & VERIFIED** Real-time API aggregation; gross revenue, pending dues, top items.	Customer aging dues breakdown, date range presets (Today, This Month, Last 30 Days).
Localization & UI (Extra Milestone)	Basic English web layout (original spec had no multi-language requirement).	**EXTRA SCOPE DELIVERED** Tri-lingual engine with English, Hindi (हिन्दी), and Gujarati (ગુજરાતી) support.	**Full i18n Suite:** Radix Select language switcher, translated table tabs, filters, and modals.
Theming & Design (Extra Milestone)	Standard light Theme B (Teal Ledger) web UI.	**EXTRA SCOPE DELIVERED** Full Dark Mode & Light Mode support with custom Teal Ledger dark surfaces.	Persistent theme storage, uniform 36px (`h-9`) header controls, accessible Radix Selects.

Module / Req	Original V1 Specification	Implementation Status & Invariants	Extra Capabilities Accomplished
SaaS Monetization (Extra Milestone)	Static tier badges (Starter, Growth, Pro).	**EXTRA SCOPE DELIVERED** End-to-end interactive checkout modal with billing cycle toggle and 18% GST calculation.	Simulated UPI/Card gateway authorization, instant plan activation, printable receipts.
Mobile Companion (Extra Milestone)	Responsive browser layout only.	**EXTRA SCOPE DELIVERED** Mobile App Companion Sync modal with QR code connection, APK download, and Expo app.	Sticky mobile bottom navigation, auto-collapsing desktop sidebar, card layout adapters.

2. Root Cause Analysis: Why Were Changes Done Piecemeal?

> **The Core Problem Identified by Leadership:** "*Currently we update things in one place only; if the same thing is used in another module we need to update that as well. For example, adding i18n languages took 3 to 4 prompts. Why wasn't that achieved in one shot?*"

An honest architectural retrospective reveals three specific technical causes for this friction:

1. **Component Dispersion (Lack of a Shared Abstraction):** Filter toolbars, search bars, and table headers were implemented individually inside module folders (e.g. `InvoiceFilterToolbar`, `PartyFilterToolbar`, `ProductFilterToolbar`) rather than consuming a unified polymorphic `<TableToolbar>` component. Local strings were hardcoded in local configs instead of binding to a global schema.
2. **Namespace Fragmentation in i18n Dictionaries:** Status keys were scattered across multiple JSON files without clear naming conventions (e.g. `invoices.json` had `filter.paid`, `common.json` had `status.paid`, while `ALL` was unmapped). When one view was translated, the others retained unmapped keys.
3. **Absence of a Mandatory Whole-App Audit Step:** AI agents previously operated with narrow "prompt-level myopia"—fixing only the screen mentioned in the user's prompt rather than grepping the codebase for all identical occurrences across Products, Invoices, Parties, Payments, Purchases, and Inventory.

3. The "One-Shot Whole-App" Engineering Standard (For Senior Dev & QA Agents)

To eliminate piecemeal edits permanently, every engineer and AI subagent must follow this non-negotiable architectural protocol:

> **Mandatory Invariant Rule: The Rule of Universal Application**
> Whenever a component pattern, UX rule, schema constraint, or translation key is modified, the developer agent MUST audit and apply the change across ALL 6 CORE DOMAINS before submitting code for QA sign-off:
> `1. Products • 2. Parties (Customers/Suppliers) • 3. Invoices • 4. Purchases • 5. Payments • 6. Inventory`

Standard Architectural Rules

- **Shared Component Enforcement:** All tables MUST use `src/components/ui/Table.tsx` and `Pagination.tsx`. All filter inputs MUST use `SearchInput.tsx` (with 300ms built-in debounce) and `Select.tsx` (Radix UI). Never write custom inline dropdowns or un-debounced inputs.
- **Universal i18n Contract:**
  - All shared statuses (`ALL`, `PAID`, `PENDING`, `OVERDUE`, `CANCELLED`, `ACTIVE`, `INACTIVE`) MUST reside in `common.json` under `status.*`.
  - All standard actions (`Create`, `Edit`, `Delete`, `Search`, `Filter`, `Reset`, `Cancel`, `Save`) MUST reside in `common.json` under `actions.*`.
  - Whenever a new string is added, the agent MUST update `en`, `hi`, and `gu` simultaneously in the same file change. Zero hardcoded strings permitted.
- **Radix UI Safety Standard:** Any dropdown or select component MUST use defensive, non-crashing components. `SelectLabel` is standardized as an accessible styled `<div>` to prevent Radix `SelectGroup` context errors.
- **Zero-Data Clean Slate Standard:** When a table has 0 records and no active search/filters, all filter bars, search inputs, and pagination controls MUST be hidden, rendering only a focused `<EmptyState>` card with a primary creation CTA.
- **Sticky Modal Architecture:** Every modal must use `<Modal footer={...}>` with full-bleed responsive buttons (full width on mobile, right-aligned on desktop). No floating dividers or nested scrollbars.

4. Comprehensive Vyapar App Decomposition

Vyapar is the market benchmark for Indian business accounting. To achieve true commercial parity, we systematically map Vyapar's complete feature architecture and phase it across disciplined release boundaries:

Vyapar Functional Pillar	Detailed Feature Capabilities	Vyora Target Version	Implementation Strategy
Sales Pipeline	Estimates / Quotations, Sale Orders, Delivery Challan, Sale Invoices, Credit Notes (Sale Return), POS counter billing.	**V1:** Sales Invoices **V2:** Estimates, Challans, Credit Notes **V3:** POS Shift Billing	V1 establishes authoritative invoice numbering and stock depletion. V2 introduces document conversion pipelines (Estimate → Invoice).
Purchase Pipeline	Purchase Orders, Purchase Bills, Debit Notes (Purchase Return), Supplier Price Lists.	**V1:** Purchase Bills **V2:** Purchase Orders, Debit Notes **V3:** Automated Reorder Requests	V1 increases physical inventory and records accounts payable. V2 adds PO tracking and return stock reversal.
Party Management & Ledger	Customer/Supplier directory, GSTIN verification, Credit Limits, Statement of Accounts, Aging Reports, WhatsApp Reminders.	**V1:** Party Master, Receivables/Payables **V2:** Credit Limits, WhatsApp Sharing **V3:** Live GSTIN Lookup API	V1 tracks net dues per party. V2 introduces PDF statement generation and direct WhatsApp notification integration.
Banking & Cash Management	Cash-in-hand, Multiple Bank Accounts, Cheque tracking, Contra transfers (Bank to Cash / Cash to Bank), Dynamic UPI QR.	**V1:** Basic Payment Methods **V2:** Bank Accounts Ledger, Contra **V3:** Auto Bank Statement Sync	V1 records payment transactions against invoices. V2 adds a true double-sided Cash/Bank ledger with dynamic invoice QR codes.
Inventory & Warehousing	Item Master, Stock Movements, Low Stock Alerts, Batch Tracking, Expiry Dates, Serial Numbers, Barcode Scanning/Printing, Multi-MRP.	**V1:** SKU, Stock Ledger, Adjustments **V2:** Batch/Expiry, Barcode Scan **V3:** Multi-Warehouse Transfers	V1 enforces FIFO atomic stock updates and low-stock indicators. V2 introduces batch-level traceability for pharma/retail.
Taxation & Compliance	GST Invoicing (CGST, SGST, IGST), HSN/SAC codes, GSTR-1, GSTR-2, GSTR-3B summaries, E-Way Bill generation, E-Invoicing (IRN).	**V1:** GST Tax Calculation, HSN **V2:** GSTR-1 & 3B JSON Exports **V3:** Live E-Way & E-Invoice API	V1 computes tax in minor units. V2 generates government-compatible JSON offline files. V3 connects to NIC/GSP sandbox APIs.
Financial Reporting	Sales/Purchase Registers, Daybook, Stock Summary, Bill-wise Profit, Profit & Loss Statement, Balance Sheet.	**V1:** Core 5 Registers **V2:** Bill-Wise Profit, Daybook **V3:** Full P&L & Balance Sheet	V1 provides operational transaction registers. V2/V3 deliver formal double-entry accounting reports.

5. Phased Product Roadmap

Version 1.0 — Core Foundation & Release Gate (Current Scope Wrap-Up)

- **Scope:** Complete daily-use billing for retail/wholesale trade. Auth, Tenant Isolation, Product Catalog, Party Directory, Sales Invoices (Draft & Finalize), Purchase Bills, Payment Vouchers (Receipt & Payment), Inventory Adjustments, Expenses, Dashboard, Core Reports.
- **Signature Polish:** Tri-lingual i18n (EN/HI/GU), Theme B Light/Dark mode, accessible Radix Selects & DatePickers, debounced search, mobile companion sync, and SaaS checkout.

Version 2.0 — Operational Parity (Vyapar Equivalent Operations)

- **Document Flow Pipeline:** Quotations / Estimates → 1-click conversion to Sales Invoices. Delivery Challans → 1-click billing.
- **Returns & Adjustments:** Credit Notes (Sales Return) and Debit Notes (Purchase Return) with automated stock replenishment.
- **Banking Ledger:** Bank Accounts & Cash In Hand ledgers; Contra vouchers; Dynamic UPI QR code printed directly on invoice PDFs.
- **Batch & Barcode:** Batch numbers, expiry date tracking, and USB/camera barcode scanner input for lightning-fast billing.

Version 3.0 — Extended Compliance, Scale & Automation

- **Tax Portal Automation:** GSTR-1 & GSTR-3B offline JSON exports; Direct E-Way Bill and E-Invoice (IRN) generation via GSP APIs.
- **Offline-First & Mobile Sync:** SQLite / WatermelonDB client database with delta-sync architecture for unreliable internet.

- **Full Accounting:** Journal vouchers, Trial Balance, Profit & Loss, and Balance Sheet verified by certified accountants.

6. Version 1 Wrap-Up Punch List (Pending UI, Schema & Fallback Gaps)

Before issuing the final V1 certification, the Senior Developer and QA agents must address and verify these specific remaining gaps:

Gap Category	Specific Issue & Target Screens	Required Resolution	Acceptance Gate
UI Action Uniformity	Some table action buttons previously rendered mixed icon/text buttons or inconsistent paddings across Payments and Purchases.	Enforce pure icon-only buttons (`<Edit2>`, `<Trash2>`, `<Eye>`) with subtle rounded backgrounds and localized tooltips.	Verified across all 6 tables in Light and Dark mode.
Clean Slate Hierarchy	When product catalog has 0 items, search bar was previously visible in certain modal views without any items to search.	Suppress search and filter controls when table has zero items in database; display prominent `<EmptyState>` card.	0-item database renders clean card with primary CTA.
Schema & Null Fallbacks	Optional notes, GSTIN, and customer addresses could display literal `"-"` or cause minor layout shifts if undefined.	Apply standardized optional chaining (`party?.gstin || t('common.na')`) and clamp text overflow with `truncate`.	No layout displacement on missing optional fields.
Inventory Adjustment DTO	Manual stock adjustment modal reason field requires strict client/server enum alignment (`DAMAGED`, `RECOUNT`, `RETURN`, `THEFT`).	Ensure frontend select options exactly match Zod schema in `@vyora/validation` and NestJS backend DTO.	Stock adjustment succeeds without validation mismatch.
100% i18n Verification	Modal titles, confirmation prompts, and toast notifications must have zero hardcoded English strings.	Audit all modal titles, buttons, toasts, and validation error messages across `en`, `hi`, and `gu`.	Switching to Hindi/Gujarati leaves 0 untranslated strings.

7. Quality Checks: The Five Layers

To enforce zero-defect releases and guarantee continuous, regression-free development across human engineers and autonomous AI agents, every change must strictly satisfy these five layers of defense:

Layer	Tool	What It Catches	Execution Command
1. Formatting	`Prettier`	Spacing, single/double quotes, line width (printWidth: 100), trailing commas, and indentation so all code looks identical regardless of editor or author.	`npm run format:check` `npm run format`
2. Linting	`ESLint + typescript-eslint`	Potential logic bugs, accidental `any` types, unused variables/imports, bad import ordering, and React hook violations (e.g. missing dependencies in `useEffect`).	`npm run lint`
3. Type Check	`tsc --noEmit (strict mode)`	Static type errors, broken interface contracts, undefined property lookups, missing return types, and schema drift between frontend and backend.	`npm run typecheck`
4. Tests	`Vitest + Testing Library`	Behavior that breaks: calculation regressions (paisa/tax math), unauthorized mutations, broken invoice finalization, and component rendering crashes.	`npm run test`
5. Gate	`Husky + lint-staged + CI`	Blocks bad code before commit and before merge. Automated pre-commit hook runs Layers 1–4 on staged files. Continuous Integration runs full suite on pull requests.	Pre-commit hook & GitHub Actions CI

8. Senior Developer Agent & QA Agent Handoff Protocol

To ensure flawless execution between autonomous AI coding agents, this strict operational workflow must be followed:

Senior Developer Agent Protocol:

1. **Atomic Vertical Slices:** Implement fixes one domain at a time (UI + validation + API + tests). Do not make sweeping unverified refactors.
2. **Whole-App Rule:** If fixing a UI pattern (e.g. icon button, modal footer, select dropdown), update all 6 modules in the same session.
3. **Satisfy Layers 1 to 4:** Format (Prettier), Lint (ESLint), Typecheck (`tsc --noEmit`), and Tests (Vitest) must all pass before handoff.
4. **Build Verification:** Always run `npm run build` in `vyora-frontend` and `npm run build` in `vyora-backend` before reporting completion. Zero TypeScript errors allowed.
5. **Strict Git Rule:** NEVER execute `git push`. Only commit locally if requested.

QA Verification Agent Protocol:

1. **Tri-Lingual Language Sweep:** Verify every page under English, Hindi, and Gujarati. Ensure table headers, filter buttons, badges, and modal titles change dynamically.
2. **Dark/Light Theme Contrast Check:** Toggle Theme B dark and light mode; verify that text contrast on table rows, card surfaces, and inputs meets WCAG AA standards.
3. **Zero-Data & Error Boundary Audit:** Verify clean empty states on empty tenants; ensure no Radix or React unhandled runtime exceptions occur.
4. **Automated Test Suite:** Execute backend integration tests (`npm run test`) and verify 100% pass rate.
5. **Gatekeeper Certification:** Validate Layer 5 pre-commit compliance before certifying release readiness.

9. Final Sign-Off & Acceptance Criteria

Verification Domain	Acceptance Criteria	Owner Sign-Off
Functional Scope	All 16 core V1 modules fully operational with real MongoDB persistence and zero mock fallbacks.	Project Manager / Lead
Quality Checks (5 Layers)	Prettier, ESLint, TypeScript (strict), Vitest test suite, and Husky pre-commit hooks passing 100%.	QA / Tech Lead
Visual & UI Consistency	Theme B (Teal Ledger) 100% consistent; h-9 header buttons; clean slate empty states; icon-only action columns.	UI/UX Designer
Localization (i18n)	Complete tri-lingual coverage (EN, HI, GU) across all pages, toolbars, tables, modals, and toasts.	QA / Localization Lead
Backend & Security	Tenant isolation on all 49 endpoints; rate limiting enabled; money in minor units; zero secrets in code.	Senior Backend Engineer